In this experiment, we implemented a character-level text generation model using a Simple RNN. The model uses an embedding layer to convert characters into dense vectors, a SimpleRNN layer with 128 units to capture sequential dependencies, and a dense softmax layer to predict the next character. However, SimpleRNN suffers from the vanishing gradient problem, making it less effective for long-term dependencies

In [11]:
```
import tensorflow as tf
import numpy as np
```

In [12]:
```
with open("sample text.txt", "r", encoding="utf-8") as f:
    text = f.read().lower()
```

In [13]:
```
chars = sorted(list(set(text)))
vocab_size = len(chars)
char_to_idx = {c: i for i, c in enumerate(chars)}
idx_to_char = {i: c for i, c in enumerate(chars)}
encoded = np.array([char_to_idx[c] for c in text])
```

In [14]:
```
seq_len = 50
X, y = [], []
for i in range(len(encoded) - seq_len):
    X.append(encoded[i:i + seq_len])
    y.append(encoded[i + seq_len])
X = np.array(X)
y = np.array(y)
```

In [15]:
```
model_rnn = tf.keras.Sequential([
    tf.keras.layers.Embedding(vocab_size, 64, input_length=seq_len),
    tf.keras.layers.SimpleRNN(128),
    tf.keras.layers.Dense(vocab_size, activation="softmax")
])
model_rnn.compile(
    loss="sparse_categorical_crossentropy",
    optimizer="adam"
)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97:
UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
```

In [16]:
```
model_rnn.fit(X, y, epochs=30, batch_size=64)
```

```
Epoch 1/30
34/34 ━━━━━━━━━━━━━━━━━━━━ 5s 35ms/step - loss: 3.2582
Epoch 2/30
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 2.9484
Epoch 3/30
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 2.8892
Epoch 4/30
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 2.7513
Epoch 5/30
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 2.5868
Epoch 6/30
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 2.4176
Epoch 7/30
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 2.2838
Epoch 8/30
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 2.2054
Epoch 9/30
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 2.0674
Epoch 10/30
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 1.9907
Epoch 11/30
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 1.9036
Epoch 12/30
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 1.8469
Epoch 13/30
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 1.7242
Epoch 14/30
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 1.6234
Epoch 15/30
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 1.5924
Epoch 16/30
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 1.5088
Epoch 17/30
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 1.4255
Epoch 18/30
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 1.3872
Epoch 19/30
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 1.2943
Epoch 20/30
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 1.1650
Epoch 21/30
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 1.1710
Epoch 22/30
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 1.1033
Epoch 23/30
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 1.0302
Epoch 24/30
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 0.9818
Epoch 25/30
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 0.9093
Epoch 26/30
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 0.8424
Epoch 27/30
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 0.8622
```

```
Epoch 28/30
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 0.8395
Epoch 29/30
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 0.7536
Epoch 30/30
34/34 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 0.7217
```

Out[16]: <keras.src.callbacks.history.History at 0x7bd66b1063f0>

In [18]:
```python
def sample_with_temperature(preds, temperature=0.8):
    preds = np.asarray(preds).astype("float64")
    preds = np.log(preds + 1e-8) / temperature
    exp_preds = np.exp(preds)
    preds = exp_preds / np.sum(exp_preds)
    return np.random.choice(len(preds), p=preds)
```

In [19]:
```python
def generate_text(seed, length=600, temperature=0.8):
    result = seed
    for _ in range(length):
        encoded_seed = [char_to_idx[c] for c in seed]
        encoded_seed = np.array(encoded_seed).reshape(1, -1)
        preds = model_rnn.predict(encoded_seed, verbose=0)[0]
        next_idx = sample_with_temperature(preds, temperature)
        next_char = idx_to_char[next_idx]
        result += next_char
        seed = seed[1:] + next_char
    return result
```

In [20]:
```python
seed_text = text[:50]
print("\nGenerated Text (Simple RNN):\n")
print(generate_text(seed_text))
```

```
Generated Text (Simple RNN):


artificial intelligence is one of the most important ovarin er ine ispeventerc
alag acale cene suatian.

re prothec  ondellayy denhmacingecal gonithis thecsused mmaron, ind inparorum
ay. arqeu
de pearnd bealning.

desparn and watalizge aguanis in araganis or, cucer isutien poriticne lragfaren
oustes af arceel orvee sored sudelsiseta d itarisig erpbece tomed bacobilg.
udd ntperatroct emanventarpigy ce bentinas
antontemtint of dumenclraitlon ncale inielligbnon
maltificlaby
ng armate ar ontuses, ard bethb artial iangurea to intel coninsven owitinis
en pitianaleanging, sedespris nis dithempons
an lihg carelsorm ag inicllingo, andates int outero n
```